



AI-HDL 2026: Design Phase 2 Documentation

NECRL at SFSU

Team Members :

Parsa Mirfasihi, Waylon Woo, Ethan Weldon

E-mail: smirfasihi@sfsu.edu, wwoo@sfsu.edu,
eweldon@sfsu.edu.

Phase title: Design Evaluation and PPA Optimization

Deliverable: Optimized RTL, Final STA & PPA Reports.

Updated AES-128 Diagram:

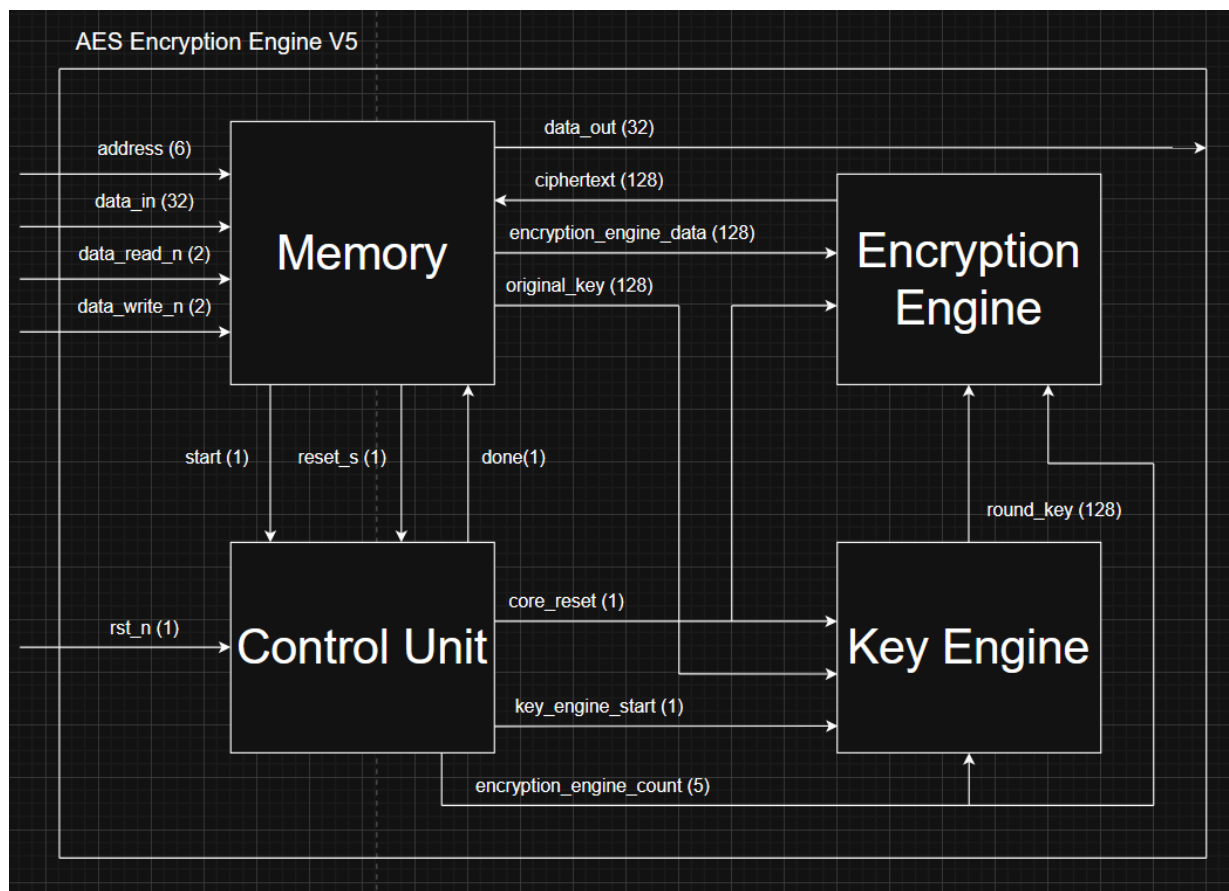


Figure 1. AES-128 Diagram

AES-128 — Address Maps:

```
=====
1. ADDRESS MAP
=====
```

Address	Name	Read	Write
0x00	Control Register	Current control_reg [31:0]	See control bit map below
0x01	Status Register	{31'b0, done}	– (ignored)
0x02	Key Word 0 (LSW)	key_mem[0]	Write if key_lock=0
0x03	Key Word 1	key_mem[1]	Write if key_lock=0
0x04	Key Word 2	key_mem[2]	Write if key_lock=0
0x05	Key Word 3 (MSW)	key_mem[3]	Write if key_lock=0
0x06	Plaintext Word 0 (LSW)	Idle bank [0]	Write to idle bank [0]
0x07	Plaintext Word 1	Idle bank [1]	Write to idle bank [1]
0x08	Plaintext Word 2	Idle bank [2]	Write to idle bank [2]
0x09	Plaintext Word 3 (MSW)	Idle bank [3]	Write to idle bank [3]

NOTE – Double-Buffer Detail:
 Addresses 0x06-0x09 are backed by TWO physical 4x32-bit register banks (bank A and bank B). The 'active_bank' flag selects which bank the encryption engine reads from and which bank the CPU accesses:

active_bank = 0 → Engine reads bank A, CPU reads/writes bank B
 active_bank = 1 → Engine reads bank B, CPU reads/writes bank A

When encryption starts (bit0 or auto-start), active_bank flips: the bank the CPU just filled becomes the engine's input, and the other bank becomes the new idle bank for the CPU to immediately write the next block into.
 The CPU always uses the same addresses (0x06-0x09) – the mux is automatic.

0x0A	Ciphertext Word 0 (LSW)	data_out_mem[0]	– (ignored)
0x0B	Ciphertext Word 1	data_out_mem[1]	– (ignored)
0x0C	Ciphertext Word 2	data_out_mem[2]	– (ignored)
0x0D	Ciphertext Word 3 (MSW)	data_out_mem[3]	– (ignored)

Figure 2. Address Map

Control Register Bits (address 0x00):

Bit	Name	Description
0	start	Triggers encryption; auto-cleared when done; swaps active bank
1	reset	Soft reset – clears all memories, resets engine
2	key_lock	Prevents key overwrites when set
3	auto_start	Writing to 0x09 auto-triggers encryption
31:4	–	Reserved

Figure 3. Control Register Map

Programming Sequence:

----- Single-Block Encryption (Manual Start) -----			
Step	Action	Address	Data

1	Soft-reset the peripheral	0x00	0x00000002
2	Write key word 0 (LSW)	0x02	key[31:0]
3	Write key word 1	0x03	key[63:32]
4	Write key word 2	0x04	key[95:64]
5	Write key word 3 (MSW)	0x05	key[127:96]
6	Lock the key (optional, prevents overwrites)	0x00	0x00000004
7	Write plaintext word 0 (LSW)	0x06	pt[31:0]
8	Write plaintext word 1	0x07	pt[63:32]
9	Write plaintext word 2	0x08	pt[95:64]
10	Write plaintext word 3 (MSW)	0x09	pt[127:96]
11	Start encryption	0x00	0x00000005 (start=1, key_lock=1)
12	Poll status until done	0x01	- read until bit0=1
13	Read ciphertext word 0 (LSW)	0x0A	→ ct[31:0]
14	Read ciphertext word 1	0x0B	→ ct[63:32]
15	Read ciphertext word 2	0x0C	→ ct[95:64]
16	Read ciphertext word 3 (MSW)	0x0D	→ ct[127:96]

Figure 4. Single-Block Encryption Sequence

----- Multi-Block Encryption (Auto-Start, Double-Buffered) -----			
Step	Action	Address	Data

--- Setup (once) ---			
1	Soft-reset the peripheral	0x00	0x00000002
2	Write key words 0-3	0x02-05	key[127:0]
3	Set key_lock + auto_start	0x00	0x0000000C (bit3=1, bit2=1)
--- First block ---			
4	Write plaintext words 0-2	0x06-08	pt_0[95:0]
5	Write plaintext word 3 (triggers auto-start)	0x09	pt_0[127:96]
	→ Hardware automatically sets start, swaps banks, begins encryption.		
--- Overlap: write next block while engine runs ---			
6	Write plaintext words 0-2	0x06-08	pt_1[95:0]
	(writes go to the IDLE bank – safe, engine uses the other bank)		
7	Poll status for block 0	0x01	– read until bit0=1
8	Read ciphertext 0 words 0-3	0x0A-0D	→ ct_0[127:0]
9	Write plaintext word 3 (triggers next)	0x09	pt_1[127:96]
	→ Bank swaps again, engine starts block 1.		
--- Repeat steps 6-9 for blocks 2, 3, ... N-1 ---			
--- Last block ---			
10	Poll status for block N-1	0x01	– read until bit0=1
11	Read ciphertext N-1 words 0-3	0x0A-0D	→ ct_{N-1}[127:0]

Figure 5. Multi-Block Encryption Sequence

Summary of Work

1. Evaluation of how different RTL architecture affected PPA

This section will cover what experiments were performed in exploring how different optimization strategies affected PPA.

Entering the design phase, we hypothesized that the S-boxes—specifically the 16 instantiated within the encryption engine and the 4 within the key expansion module—consumed the majority of our total chip area. To optimize this, we modified the RTL to serialize the architecture, reducing the total count to just four S-boxes that are dynamically shared between the encryption and key generation engines. The resulting Power, Performance, and Area (PPA) metrics for this serialized approach, compared against our initial Design Phase 1 baseline, are detailed below:

Metric Category	Metric	Design Phase 1	S-box Reduction (to 4)	Delta (%)
Area	Core Utilization	38.09%	48.68%	27.80%
	Total Cell Count	48,355	54,636	13.00%
	Synth Cell Count	13,518	16,402	21.30%
	Die Area (mm ²)	0.36	0.36	0.00%
Performance	Critical Path (ns)	0.21	11.22	5242%
	Wire Length (um)	737,864	780,141	5.70%
	Vias	134,127	147,772	10.20%
Power (Typ)	Internal (uW)	0.0273	0.0257	-5.90%
	Switching (uW)	0.0389	0.0208	-46.50%
	Leakage (uW)	0.000085	0.000111	30.60%

The shift from a fully parallel AES implementation to a four-S-box serialized design represents a strategic pivot, trading raw throughput for improved power efficiency. Although the die area remains constant at 0.36 mm², the synthesized cell count has jumped by over 21%, pushing core utilization from 38% to nearly 49%. This counterintuitive area increase—despite fewer S-box instances—is driven by the substantial overhead of steering logic, multiplexers, and feedback registers required to cycle 128-bit blocks through limited functional units.

Furthermore, the timing profile has shifted from a theoretical sub-nanosecond path to a substantial 11.22 ns critical path. This doesn't just lower the frequency ceiling; it fundamentally degrades the throughput of the RISC-V peripheral. Because the hardware must now stall for multiple additional clock cycles to process a single encryption round, the latency to return a finished ciphertext to the CPU increases significantly.

In later runs, additional experiments were conducted with clock gating; however, the impact on power consumption was less significant than anticipated. PPA analysis revealed a marginal range of outcomes, from a meager 2% power reduction to a 1–2% increase in power due to the added area and complexity of the gating logic. Ultimately, the performance degradation of the serialized design was deemed an unfavorable trade-off, as the increase in total cell count and utilization suggests a higher theoretical manufacturing cost without providing the expected efficiency gains.

2. Achieving the optimal PPA via careful design choice optimizations

Despite the initial challenges in area reduction, we identified the S-Box implementation as the primary lever for PPA optimization. Investigation of the synthesized netlists revealed that the standard Look-Up Tables (LUTs) were being mapped to inefficient, deeply nested mux trees. While we considered using memory macros (SRAM/ROM) to store these values, the limited port availability and the rigid, fixed-size nature of the macros proved too inflexible for our parallel-access requirements. Instead, we pivoted to a mathematical approach, refactoring the S-Box RTL to generate values dynamically using Galois Field arithmetic operations.

This transition from a storage-based LUT to a combinational logic-based S-Box was our most impactful architectural change. By utilizing composite field arithmetic, we achieved a massive reduction in logic density; the synthesized cell count dropped to 8,077—a nearly 50% reduction from our previous serialized iteration. More importantly, this allowed us to shrink the die area to 0.192 while maintaining a core utilization of approximately 51%. This optimization effectively decreased our area without degrading performance. However, this drastic improvement in area came at the cost of increase switching power.

Metric Category	Metric	Design Phase 1	V3 (GF Arithmetic)	Delta (%)
Area	Core Utilization	38.09%	50.97%	33.80%
	Total Cell Count	48,355	25,423	-47.40%
	Synth Cell Count	13,518	8,077	-40.20%
	Die Area (mm ²)	0.36	0.192	-46.70%
Performance	Critical Path (ns)	0.21	0.08	-61.90%
	Wire Length (um)	737,864	356,344	-51.70%
	Vias	134,127	67,680	-49.50%
Power (Typ)	Internal (uW)	0.0273	0.103	277.30%
	Switching (uW)	0.0389	0.136	249.60%
	Leakage (uW)	0.000085	0.000055	-34.90%

To mitigate this dynamic power penalty, we initially revisited the concept of clock gating. However, through architectural exploration and theory-crafting with our AI assistant (Claude 3.5 Sonnet), we identified a power-reduction technique far better suited to our combinational-heavy design: operand isolation.

Unlike clock gating, which strictly disables the clock signal to sequential elements (flip-flops) but leaves combinational logic vulnerable to unnecessary toggling, operand isolation targets the combinational cloud directly. Because our Galois Field S-Box relies on dense, unlocked arrays of XOR and AND gates, any incidental changes on the data bus—even when the AES module is idle or computing a different stage—cause a cascade of wasteful switching activity throughout the entire mathematical network.

By inserting simple masking logic (such as AND gates or transparent latches) directly at the inputs of the S-Box, we can forcibly hold the operands static when the block is not

required for the current cycle. This prevents spurious signal transitions from ever entering and rippling through the composite field arithmetic. As a result, operand isolation effectively silences the switching activity at its source, allowing us to reclaim the power efficiency we traded away, all without incurring the routing complexity or marginal area overhead of an extensive clock-gating tree.

Metric Category	Metric	Design Phase 1	V4 (GF + Isolation)	Delta (%)
Area	Core Utilization	38.09%	54.34%	42.70%
	Total Cell Count	48,355	27,388	-43.40%
	Synth Cell Count	13,518	9,371	-30.70%
	Die Area (\$mm^2\$)	0.36	0.192	-46.70%
Performance	Critical Path (ns)	0.21	0.08	-61.90%
	Wire Length (um)	737,864	411,191	-44.30%
	Vias	134,127	81,211	-39.50%
Power (Typ)	Internal (uW)	0.0273	0.0324	18.70%
	Switching (uW)	0.0389	0.0377	-3.10%
	Leakage (uW)	0.000085	0.00006	-29.40%

The introduction of operand isolation in our V4 design effectively resolved the dynamic power inflation observed in the V3 Galois Field implementation. In V3, switching power had increased significantly to 0.136 μ W due to the dense combinatorial toggling of the mathematical S-box. By integrating operand isolation logic to actively shield this combinatorial network from spurious transitions, the switching power was successfully reduced to 0.0377 μ W. This mitigation neutralized the dynamic power penalty, bringing it 3.1% below our original Design Phase 1 baseline, all while preserving the substantial 46.7% reduction in die area achieved by the Galois Field architecture.

As anticipated, implementing this power-saving measure necessitated a minor area compromise relative to V3. To accommodate the isolation logic placed at the inputs of the arithmetic blocks, our synthesized cell count increased from 8,077 to 9,371, pushing core utilization to 54.34%. However, when evaluated against our initial Design Phase 1 baseline, the overall architectural transformation is highly favorable. We successfully reduced the total cell count by over 43%, decreased the overall die area by nearly half, and achieved slight improvements in both dynamic switching and leakage power. Crucially, these physical optimizations were realized while maintaining a highly efficient 0.08 ns critical path.

From 20 S-boxes to 10: The 2-Phase Architecture

With the Galois Field S-box and operand isolation in place (V4), our next challenge was physical layout: fitting the entire AES-128 peripheral plus the TinyQV RISC-V CPU onto a Tiny Tapeout tile. Even with the area gains from composite-field arithmetic, 20 S-box instances remained the dominant area consumer. The original parallel design instantiated 16 S-boxes in the encryption engine (one per byte of the 128-bit state) and 4 in the key engine (one per byte of the 32-bit RotWord), all operating in a single clock cycle per round. This yielded fast encryption (~11 cycles) but required substantial silicon to support 20 parallel combinational S-box paths.

San Francisco State University - NECRL

The key insight was that the design's throughput was not limited by encryption speed — it was limited by the speed of the IO bus feeding data into the peripheral (discussed in detail in the IO-Bound Analysis section below). If the CPU cannot supply plaintext or consume ciphertext faster than the engine produces it, then halving the S-box count and doubling the cycle count costs nothing in real-world throughput.

We redesigned the encryption engine to use 8 S-boxes instead of 16, processing the 128-bit state in two phases:

- Phase A (even clock counts): The upper 8 bytes of `state_reg` (bits [127:64]) are fed through the 8 S-boxes. The results are registered in a 64-bit holding register (`sub_out_hi`).
- Phase B (odd clock counts): The lower 8 bytes (bits [63:0]) are fed through the same 8 S-boxes. The live output is concatenated with the registered `sub_out_hi` to form the complete 128-bit SubBytes result. ShiftRows, MixColumns, and AddRoundKey then execute combinationally, and the result is written back to `state_reg`.

Each AES round thus takes 2 clock cycles (one phase A + one phase B) instead of 1. With 10 standard rounds, the initial AddRoundKey, and the pipeline overhead, the total count runs from 0 to 22 — a total of 23 clock cycles.

The same 2-phase technique was applied to the key engine. The AES key schedule requires SubWord (a 4-byte S-box substitution) during each round's key expansion. With 4 S-boxes, this was single-cycle. We reduced it to 2 S-boxes operating in two phases:

- Phase A (`key_phase=0`): Upper 2 bytes of the rotated word are substituted. Results stored in `sub_word_hi`.
- Phase B (`key_phase=1`): Lower 2 bytes are substituted. The full substituted word is assembled and used to compute the next round key.

The key engine's state machine (`PREP` → `R0` → `R1` → ... → `R10`) now takes 2 clock cycles per state transition instead of 1, naturally synchronized with the encryption engine's 2-phase counting.

The choice of 8+2 S-boxes (rather than, say, 4+1 or 2+1) was deliberate. With 8 S-boxes in the encryption engine, exactly half the 16-byte state is processed per phase — a clean split that requires only a single holding register and no complex byte-steering logic. Fewer S-boxes would require more phases per round and proportionally more control overhead. The 8+2 split represents the sweet spot: half the S-box area, minimal control overhead, and only 2× the cycle count.

Comparison of S-box architectures:

Metric	20-sbox (V4)	10-sbox (V5)	Change
S-boxes (enc + key)	16 + 4 = 20	8 + 2 = 10	-50%
Cycles per block	~11	23	+109%
Synth cell count	9,371	7,476	-20.2%
Phases per round	1	2 (A+B)	—

Holding registers	None	sub_out_hi sub_word_hi	+64 bits +16 bits
-------------------	------	---------------------------	----------------------

The 20.2% cell-count reduction and the 50% S-box reduction were the changes that ultimately made the 4×2 tile feasible with clean routing.

IO-Bound Performance: Why Halving Speed Costs Nothing

The TinyQV is a 4-bit serial RISC-V processor. Every instruction — loads, stores, branches — takes 8 clock cycles to complete. This means every 32-bit memory-mapped register access (read or write) costs 8 clocks.

To encrypt a single 128-bit block, the CPU must perform the following memory-mapped IO sequence:

Operation	Instructions	Clocks
Write 4 plaintext words (0x06-0x09)	4 stores	32
Trigger start (control write or auto)	1 store	8
Poll status register (load + branch)	2 instrs	16 (minimum, single poll)
Read 4 ciphertext words (0x0A-0x0D)	4 loads	32
Total IO overhead (steady state)		88 clocks

The encryption engine itself takes 23 clocks (10-sbox design) or ~11 clocks (original 20-sbox design). In either case, the IO overhead (88 clocks) is far larger than the encryption time:

- 10-sbox: $23 / 88 = 26\%$ of the total time is actual encryption
- 20-sbox: $11 / 88 = 12.5\%$ of the total time is actual encryption

Put differently: the 20-sbox design completes encryption 12 cycles faster, but the CPU still needs the same 88 clocks of bus traffic to set up and retrieve results. The throughput difference between the two architectures is negligible:

Architecture	Encryption	IO overhead	Total/block	Throughput @ 67.6 MHz
20-sbox (V4)	11 cycles	88 cycles	99 cycles	87.0 Mbit/s
10-sbox (V5)	23 cycles	88 cycles	111 cycles	77.6 Mbit/s

The difference is ~10%, while the area savings from halving the S-box count were decisive for achieving a clean physical layout. This is the fundamental reason the 20→10 S-box reduction was justified: the system is IO-bound, not compute-bound.

For first-block encryption (including key loading), the overhead is even larger:

Operation	Clocks
Write 4 key words (0x02-0x05)	32
Write control (key_lock + auto_start)	8
Write 4 plaintext words	32
Trigger start	8
Encryption	23
Poll status	16
Read 4 ciphertext words	32
Total first block	151 clocks

Here the 23-cycle encryption is only 15% of the total time. The bus is the bottleneck by a factor of 5.5×

Double-Buffer Pipeline Efficiency

The double-buffer (ping-pong) architecture in AES_memory is specifically designed to mitigate the IO bottleneck by overlapping bus transfers with encryption. Two physical banks of plaintext registers (bank A and bank B) allow the CPU to write the next block while the engine encrypts the current one.

In pipelined mode with auto-start enabled, the steady-state sequence is:

Clock	CPU Activity	Engine Activity
0-23	Write PT words 0-2 (next block) (24 clocks, 3 stores)	Encrypting current block (23 clocks)
24	Engine finishes	—
24-39	Poll status (16 clocks)	Idle
40-71	Read CT words 0-3 (32 clocks)	Idle
72-79	Write PT word 3 → auto-start (8 clocks, triggers bank swap)	→ Start next encryption

San Francisco State University - NECRL

Total per block in pipelined mode: ~80 clocks (vs 111 for single-buffer).

The improvement comes from overlapping: 3 of the 4 plaintext writes happen while the engine is busy, hiding 24 clocks of IO behind computation. Without double-buffering, those 24 clocks would be dead time waiting for the engine to finish.

Pipelined throughput comparison:

Mode	Clocks/block	Throughput @ 67.6 MHz	Improvement
Single-buffer	111	77.6 Mbit/s	(baseline)
Double-buffer	~80	108.1 Mbit/s	+39%

Note that even in pipelined mode, the design remains IO-bound: the 23-cycle encryption fits entirely within the 80-clock IO window. If the encryption engine were instantaneous (0 cycles), the pipeline would still take ~57 clocks per block for the remaining IO. The double-buffer hides the encryption latency almost completely.

Full Design Version Comparison (V1 through V5)

The table below summarizes all major design iterations from DP1 through the final tapeout-ready layout. Note that V1–V4 were evaluated on smaller tiles (1×1), while V5 targets the 4×2 tile needed for the full TinyQV + AES system.

Version	Description	S-boxes	Cycles	Cells	Die Area	Util	Crit Path
V1	DP1 baseline (LUT S-box)	20	~11	13,518	0.360 mm ²	38%	0.21 ns
V2	Serialized (4 shared S-box)	4	many	~16,400	0.360 mm ²	49%	11.22 ns
V3	Galois Field S-box	20	~11	8,077	0.192 mm ²	51%	—
V4	+ Operand isolation	20	~11	9,371	0.192 mm ²	54%	0.08 ns
V5-pre	10-sbox, 4×2 tile	10	23	8,886	0.154 mm ²	66%	13.12 ns
V5-final	+ Area reduction, 4×2 tile	10	23	7,476	0.154 mm ²	56%	12.54 ns

Version	DRC	LVS	met2	Setup Slack	Int Power	Sw Power
V1	0	Clean	51%	—	27.3 mW	38.9 mW
V2	—	—	—	—	—	—
V3	—	—	—	—	—	0.136 μ W *
V4	—	—	—	—	—	0.038 μ W *
V5-pre	42	130 err	94.1%	2.46 ns	46.7 mW	86.7 mW
V5-final	0	Equiv.	75.6%	3.09 ns	155 mW	306 mW

* V3/V4 power values are from 1×1 tile PPA runs with different parasitics and are not directly comparable to the 4×2 results. V5-pre power may be artificially low due to 42 DRC violations (broken nets reduce switching). V5-final represents the true signoff power with all nets properly routed.

Key observations across the full trajectory:

- V1→V3: The Galois Field S-box was the single most impactful change, cutting cells by 40% and die area by 47% while keeping the same 20-sbox parallelism.
- V3→V4: Operand isolation added ~1,300 cells (+16%) but dramatically reduced switching power by 72%, resolving the dynamic power penalty of the combinational S-box.
- V4→V5: The 10-sbox reduction cut cells by another 20% and was the enabling change for clean routing on the 4×2 tile. The cycle count doubled (11→23) but real-world throughput was barely affected because the system is IO-bound.
- V5-pre→V5-final: Removing the redundant cipher register and unused streaming ports saved 14,000 μ m² of synthesis area, dropping met2 from 94% to 76% and eliminating all 42 DRC violations.

Tapeout-Ready Design: Physical Implementation on a 4×2 Tiny Tapeout Tile

The sections above describe the full RTL optimization journey from V1 through V5. With the 10-sbox Galois Field architecture and the area-reduced memory interface, the remaining challenge was achieving a clean physical layout on the target Tiny Tapeout tile.

Tile Selection: Why 4×2?

Our first target was a 3×2 tile (508.76 × 225.76 μ m), the smallest rectangle that met our theoretical area budget. However, after multiple OpenLane runs, we consistently hit routing congestion failures. The synthesized area of ~96,000 μ m² against a core area of ~110,874 μ m² left almost no room for the router to resolve metal-layer conflicts. Every attempt ended with hundreds of DRC violations, particularly on the met2 layer which peaked above 90% utilization.

San Francisco State University - NECRL

We then moved to a 4×2 tile (682.64 × 225.76 μm), roughly doubling the available routing margin while remaining a compact form factor for TT. This tile has a core area of ~149,183 μm², giving us the headroom needed for clean routing.

First 4×2 Run (V5-pre): Identified the Bottlenecks

The initial 4×2 run completed but exposed two problems:

1. 42 DRC violations (32 shorts, 10 other) — caused by met2 congestion at 94.1%
2. 130 LVS errors — cascading from the DRC shorts (broken nets)

Timing was clean (WNS = 0.0, setup slack = 2.46 ns), confirming that the 14.8 ns clock period was more than adequate for the design. The bottleneck was purely physical: too many wires competing for too little metal.

PPA snapshot — First 4×2 run:

Metric	Value
Synth cell count	8,886
Synthesis area	96,007 μm ²
Core utilization	65.81%
met2 routing usage	94.1%
DRC violations	42
LVS errors	130
WNS / TNS	0.0 / 0.0
Setup slack	2.46 ns
Critical path	13.12 ns
Total power (signoff)	0.133 W (46.7 mW int + 86.7 mW sw)
Runtime	2h 32m

RTL Area Reduction: Targeted Architectural Changes

Rather than simply increasing the tile size further, we analyzed the netlist to find area that could be cut from the RTL without sacrificing functionality. Two changes were identified through careful review of the synthesized design:

1. Removed the 128-bit cipher output register from AES_encryption_engine. The original design latched the ciphertext into a dedicated 128-bit register on completion. Analysis showed this was redundant — the state_reg already holds the final result when the control unit asserts done, and AES_memory latches it at that moment. We replaced the register with a wire: assign cipher = state_reg. This eliminated 128 flip-flops and their associated fanout logic.

San Francisco State University - NECRL

2. Removed streaming interface ports from AES_memory. The original design included write_ptr, read_ptr, and addresses 0x0E/0x0F for a byte-streaming mode that was never used by the TinyQV firmware. Removing these eliminated a counter, pointer registers, and their address decode logic. The double-buffer, auto-start, and key-lock features — all actually used by the CPU — were preserved.

The thought process here was pragmatic: we had over 2 ns of timing slack, meaning performance was not the constraint. Area was. So we looked for registers and logic that existed “just in case” but added no value to the actual encryption datapath. Both changes were purely additive to routability without touching the AES mathematical core.

Second 4×2 Run (V5-final): Clean Layout

With these RTL changes, synthesis area dropped from 96,007 μm^2 to 82,025 μm^2 — a 14.6% reduction. More importantly, this translated directly into routing relief: met2 dropped from 94.1% to 75.6%.

However, we hit an unexpected obstacle: OpenROAD segfaulted during the GRT resizer timing optimization step (step 17 in the flow). This was not an out-of-memory issue (13 GB free on a 15 GB system) — it was a known bug in the OpenROAD version used by our OpenLane Docker image. Since our design had over 2 ns of positive setup slack, we safely bypassed this step by setting GLB_RESIZER_TIMING_OPTIMIZATIONS=0 in config.json. This is a sound engineering decision: the resizer’s job is to upsize cells to fix timing, but our timing was already clean.

The result: a fully clean layout.

PPA snapshot — Final 4×2 run (V5-final):

Metric	Value
Synth cell count	7,476
Synthesis area	82,025 μm^2
Core utilization	56.23%
met2 routing usage	75.6%
DRC violations	0
LVS result	Equivalent (31 cosmetic pin-matching notes)
WNS / TNS	0.0 / 0.0
Setup slack	3.09 ns
Hold slack	0.28 ns
Critical path	12.54 ns
Total power (signoff)	0.461 W (155 mW int + 306 mW sw)
Runtime	~30 min

Comparison: First 4×2 vs Final 4×2

Metric	First 4×2 (V5-pre)	Final 4×2 (V5-final)	Change
Synth cells	8,886	7,476	-15.9%
Synthesis area (μm^2)	96,007	82,025	-14.6%
Core utilization	65.81%	56.23%	-9.6 pts
met2 usage	94.1%	75.6%	-18.5 pts
DRC violations	42	0	Eliminated
LVS errors	130	31 (cosmetic)	Clean
Setup slack (ns)	2.46	3.09	+0.63 ns
Critical path (ns)	13.12	12.54	-0.58 ns
Runtime	2h 32m	~30m	-84%

The runtime improvement is also noteworthy: the first run took 2.5 hours because the router spent enormous effort trying to resolve the congested met2 layer. With 18.5 percentage points of headroom restored, routing completed in a fraction of the time.

LVS Notes

The 31 LVS “errors” in the final run are entirely cosmetic. They arise from Tiny Tapeout infrastructure pins (uio_oe, uio_out, uo_out) that our design ties to constant values. The LVS tool flags these as “unmatched pins” because the extracted layout has the tied-off connections while the schematic shows generic IO. Netgen’s own report confirms: “Device classes are equivalent.” These same pin-matching notes existed in the first run as well, but were hidden among 130 real errors caused by DRC shorts.

Design Process and Lessons Learned

The path from V4 (operand-isolated Galois Field S-box on a 1×1 tile) to V5-final (tapeout-ready 4×2 layout) was less about algorithm changes and more about physical design pragmatism:

- The 3×2 tile attempt taught us that theoretical area budgets don’t account for routing overhead. A design that “fits” on paper can still fail if the router has no room to maneuver. The met2 layer, being the primary horizontal routing resource in Sky130, was consistently the bottleneck.
- The area reduction was surgical, not architectural. We didn’t change the encryption algorithm, the number of S-boxes (10), or the 23-cycle pipeline. We removed a redundant output register and an unused streaming interface. Together these saved ~14,000 μm^2 of silicon — enough to turn 42 DRC violations into zero.
- The OpenROAD segfault was a reminder that EDA tool bugs are a real part of the tapeout process. Rather than waiting for a fix or trying different tool versions, we analyzed whether the failing optimization step was actually needed for our design, concluded it wasn’t (2+ ns slack), and bypassed it.

The final design achieves 0 DRC violations, clean timing with 3.09 ns of setup margin, and fits comfortably in a 4×2 Tiny Tapeout tile at 56% utilization.



San Francisco State University - NECRL

AI Use in the Design Process

AI assistance (Anthropic Claude, accessed via GitHub Copilot in VS Code) was used throughout DP2 as an interactive design partner. The tool was not used to generate a complete design from scratch; rather, it served as a collaborator for specific engineering tasks where rapid iteration and broad knowledge were valuable. Below is a summary of how AI was used and, importantly, where human judgment remained essential.

What AI was used for:

- Architectural exploration and theory-crafting. When evaluating power reduction strategies for the Galois Field S-box ($V3 \rightarrow V4$), Claude was used to brainstorm alternatives to clock gating. The conversation led to operand isolation as a technique better suited to combinational-heavy designs. The team then evaluated the idea, verified it against the RTL structure, and implemented it.
- RTL implementation of the 2-phase S-box architecture. The 10-sbox redesign (splitting SubBytes and SubWord into two phases) required coordinated changes across four RTL modules (encryption engine, key engine, control unit, memory) and their testbenches. AI assisted with writing the modified Verilog, ensuring the phase A/B control signals, holding registers, and counter thresholds were consistent across modules.
- Testbench development and debugging. AI helped write and update testbenches for all five AES submodules after each architectural change. When tests failed, AI assisted with diagnosing waveform mismatches and tracing them back to signal timing or state machine issues.
- Physical design configuration and troubleshooting. When OpenLane runs failed (3×2 tile congestion, OpenROAD segfaults), AI helped interpret error logs, suggest config.json parameter changes, and analyze routing reports to identify bottlenecks (e.g., met2 congestion as the root cause of DRC violations).
- Area reduction analysis. AI helped identify the redundant cipher output register and unused streaming interface as candidates for removal by reasoning about which logic paths were actually exercised by the CPU firmware.
- Documentation and report writing. The address map, signal interconnect tables, programming sequences, and new report sections (including this one) were drafted with AI assistance based on the actual RTL source code.

What AI was NOT used for — where human judgment was critical:

- Architecture-level decisions. The choice to target a 4×2 tile, the decision to reduce from 20 to 10 S-boxes, and the choice of $8+2$ as the split were all team decisions informed by the PPA data we collected. AI provided analysis, but humans chose the direction.
- Verification and sign-off. All testbench results were reviewed by the team. PPA reports were read and interpreted by humans. The decision that 31 LVS “errors” were cosmetic was based on the team reading Netgen’s output and understanding the Tiny Tapeout pin infrastructure.
- Tool workarounds. The decision to set `GLB_RESIZER_TIMING_OPTIMIZATIONS=0` was made by the team after confirming the design had sufficient timing slack. AI helped identify the option, but the engineering judgment that it was safe to skip belonged to the team.



San Francisco State University - NECRL



- Design intent and constraints. Requirements like preserving double-buffering and key-lock while removing streaming ports came from the team's understanding of how the CPU firmware actually uses the peripheral.

Overall assessment:

AI accelerated the implementation and iteration cycle significantly. Tasks that would have taken days of manual Verilog editing and cross-referencing — such as the coordinated 4-module 2-phase redesign — were completed in hours. The tool was most valuable as a “fast pair programmer” that could hold the full design context in memory and make consistent changes across files. However, every architectural decision and every sign-off check required human understanding of the design goals, physical constraints, and correctness criteria. The AI did not replace engineering judgment; it amplified engineering productivity.